

Lesson 11

Portfolio Polish, Security & Deployment

Code the Dream — React

Today's Session

EXPLORE

- What makes a project portfolio-ready
- Styling approaches for React apps
- Code cleanliness and README structure
- XSS and React's built-in protection
- Environment variables — what's safe
- Client-side storage and dependency security

APPLY

- Style your app professionally
- Add input sanitization with DOMPurify
- Write a comprehensive README
- Deploy to Vercel (optional)

By the end: your React app is portfolio-ready — styled, documented, secured, and optionally live on the web.

What Makes a Project Portfolio-Ready

Employers evaluate three things — not just whether the app works:

Visual presentation — first impression matters

- Professional styling, responsive design, consistent color scheme
- Loading states, error states, and empty states handled gracefully

Code quality — credibility as a developer

- Clean organization, consistent naming conventions
- No debug code, unused imports, or `console.log` statements

Documentation — communication skills

- A README that tells the project's story
- What you built, what you learned, what you'd add next

Styling Approaches for React

Choose one and apply it consistently throughout your app:

Approach	Best for	Notes
CSS Modules	Familiarity with CSS	Already included in Vite — no setup needed
Tailwind CSS	Rapid development	Utility classes; consistent design system
Styled-components	Dynamic, props-based styles	CSS-in-JS; <code>npm install styled-components</code>
Component library (MUI, Ant Design)	Pre-built accessible components	Steeper learning curve but production-quality

CSS Modules is the lowest-friction starting point — it uses regular CSS with automatic scoping so class names don't conflict across components.

Code Cleanliness — Pre-Portfolio Checklist

Before you share your repo link with anyone, run through this:

Remove:

- `console.log` and debug statements
- Commented-out code blocks
- Unused imports and variables
- Dead code and unused components

Verify:

- Component names are `PascalCase`, files match component names
- Functions and variables are `camelCase` with descriptive names
- One component per file
- ESLint shows no errors; Prettier has been run

Code Cleanliness (Continued) — Comments

Comments:

- Only add comments that explain the *why* — not the *what*
- Remove any TODO comments you don't plan to address

README Structure That Gets Read

A strong README tells your project's story. Include:

- **Title + description** — one clear sentence on what the app does
- **Live demo link** — the most important line in the file
- **Screenshots** — desktop and mobile views
- **Technologies used** — React, Vite, React Router, etc.
- **Features** — bulleted list of what users can do
- **Installation** — clone → install → run dev server
- **Your development story** — challenges you solved, what you learned
- **Future improvements** — features you'd add with more time

The "development story" section is what separates your README from a template. Employers want to know how you think — not just what you built.

XSS and React's Built-In Protection

XSS (Cross-Site Scripting): an attacker injects malicious script that runs in another user's browser, potentially stealing data or hijacking sessions.

React automatically escapes values rendered in JSX — this prevents most XSS:

```
// Safe – React renders this as text, not executable code
<p>{userComment}</p>
// Even "<script>alert('xss')</script>" is displayed safely as text
```

The danger appears only when you bypass React's escaping:

```
// DANGEROUS – executes whatever HTML string is passed in
<div dangerouslySetInnerHTML={{ __html: userComment }} />
```

Fix: sanitize with **DOMPurify** before inserting any HTML:

```
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(content) }} />
```


Input Validation — Client vs. Server

Client-side validation = user experience. Immediate feedback that helps users fill forms correctly.

Server-side validation = security. The real gate. Cannot be bypassed.

```
// Client: shows error to help the user
if (title.trim() === '') {
  setError('Title is required');
  return;
}
```

```
// BUT: users can bypass client validation by:
// - Disabling JavaScript
// - Modifying code in DevTools
// - Sending HTTP requests directly to the API
```

Never treat client-side validation as a security measure. It is a UX improvement. The server must validate every request independently, regardless of what the client sent.

Environment Variables — What's Safe

In Vite, only variables prefixed with `VITE_` are included in the client bundle — and therefore visible to all users.

Safe to use in React (<code>VITE_</code>)	Never in React code
<code>VITE_API_BASE_URL</code>	<code>DATABASE_URL</code>
<code>VITE_APP_NAME</code>	<code>JWT_SECRET</code>
<code>VITE_STRIPE_PUBLISHABLE_KEY</code>	<code>STRIPE_SECRET_KEY</code>
<code>VITE_ANALYTICS_ID</code>	<code>SENDGRID_API_KEY</code>

Access in components: `import.meta.env.VITE_API_BASE_URL`

The built JavaScript bundle is downloaded by every user. Anyone can open DevTools and search it. There is no hiding or obfuscating a secret in client-side code.

Never Store Secrets in Client Code

The correct pattern: React calls *your* API; your server holds the secrets.

```
// WRONG – secret key visible to every user in the bundle
const resp = await fetch('https://api.sendgrid.com/v3/mail/send', {
  headers: { 'Authorization': `Bearer ${SECRET_KEY}` },
});

// CORRECT – React calls your backend; backend holds the key
const resp = await fetch('/api/send-email', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${userToken}`, // user auth, not service key
  },
  body: JSON.stringify({ to: recipient, message }),
});
```

Your `/api/send-email` endpoint uses the secret key server-side, where it can't be inspected.

Check for Understanding

Which of these is safe to put in a Vite React app's `.env` file with the `VITE_` prefix?

- A) `VITE_STRIPE_SECRET_KEY=sk_live_51H123...` — your Stripe secret key for charging cards
- B) `VITE_API_BASE_URL=https://api.myapp.com` — the base URL your app fetches from
- C) `VITE_JWT_SECRET=super_secret_signing_key` — the key used to sign authentication tokens
- D) `VITE_DATABASE_URL=postgres://user:pass@host/db` — your database connection string

Client-Side Storage — Do's and Don'ts

All browser storage is readable by any JavaScript on your domain — including injected malicious scripts.

Safe to store

```
localStorage.setItem('theme', 'dark');
localStorage.setItem('language', 'en');
sessionStorage.setItem('activeTab', 'profile');

// Public, re-fetchable data
localStorage.setItem('productList',
  JSON.stringify(products));
```

Never store

```
// These expose user data to XSS attacks
localStorage.setItem('password', pwd);
localStorage.setItem('ssn', '123-45-6789');
localStorage.setItem('apiSecret', key);
localStorage.setItem('creditCard', num);
```

Dependency Security

Every `npm install` brings in third-party code — including code from *its* dependencies. Most React projects have hundreds of transitive packages.

Audit for known vulnerabilities:

```
npm audit                # shows vulnerability report
npm audit fix            # auto-fixes what it can
npm outdated            # shows packages with newer versions
```

Before adding a new package, check:

- Weekly download count — high numbers mean more community vetting
- Last publish date — abandoned packages don't get security patches
- Maintainers — is this a known organization or an unknown individual?
- What permissions it requests

Now: Apply

Let's get your app ready for the world.

This Week's Assignment — Four Parts

Part	Task	Required?
1	Style your app professionally	Yes
2	Add DOMPurify input sanitization	Yes
3	Write a comprehensive README	Yes
4	Deploy to Vercel	Optional

This is the final assignment. Submit a **project URL** (deployed app + GitHub repo link) instead of a pull request.

Deployment is optional — students who prefer not to deploy can still complete Parts 1–3 and submit the GitHub link. Mentors may not be able to troubleshoot other deployment platforms.

Part 1 — Style Your App

Choose a styling approach and apply it consistently across all components.

What to implement:

- Consistent color scheme and spacing
- Typography hierarchy — headings, body text, labels
- Responsive layout — test in Chrome DevTools at mobile widths
- Loading state — spinner or skeleton while data fetches
- Error state — user-friendly message when something fails
- Empty state — message when the todo list is empty
- Focus indicators — visible outline for keyboard navigation
- Hover transitions on buttons and interactive elements

CSS Modules is the easiest starting point — create a `Component.module.css` file alongside each component and import it as `styles`.

Part 2 — Input Sanitization with DOMPurify

Install DOMPurify, then sanitize user input before processing or storing it.

```
npm install dompurify
```

```
import DOMPurify from 'dompurify';
```

```
function sanitizeInput(input) {  
  return DOMPurify.sanitize(input.trim(), {  
    ALLOWED_TAGS: [], // strip all HTML tags  
    ALLOWED_ATTR: [], // strip all attributes  
  });  
}
```

```
// Use before submitting  
function handleSubmit(e) {  
  e.preventDefault();  
  if (!isValidTodoTitle(workingTitle)) return;  
  const safe = sanitizeInput(workingTitle);  
  addTodo(safe);  
}
```

APPLY

Part 2 — Continued

Also verify: no secrets in `.env`, `VITE_` prefix only on public values, client-side validation on all form inputs.

Part 3 — Write Your README

Your README is the first thing a hiring manager reads. Include every section:

- **Project title + one-sentence description**
- **Live demo link** ← most important line
- **Screenshots** — desktop and mobile views
- **Technologies used** — React, Vite, React Router, your styling choice
- **Features** — bulleted list of what users can do
- **Installation** — `git clone` → `npm install` → `npm run dev`
- **Your development story** — one challenge, how you solved it, what you learned
- **Future improvements** — 2–3 ideas for what you'd add next

The "development story" is what separates your README from a template. A specific challenge + specific solution shows problem-solving skills that code alone can't communicate.

Part 4 — Deploy to Vercel (Optional)

Before deploying:

1. `npm run build` — confirm the build succeeds
2. `npm run preview` — test the production build locally

Create `vercel.json` in your project root to forward API calls:

```
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://ctd-learns-node-l42tx.ondigitalocean.app/api/:path*"
    }
  ]
}
```

Part 4 — Continued

Deploy:

1. Push all changes (including `vercel.json`) to GitHub
2. Sign up at vercel.com with your GitHub account
3. Import your repository — Vite is auto-detected
4. Add environment variables in Vercel's dashboard
5. Click Deploy → copy your live URL into your README

Congratulations — You Did It!

Over 11 weeks you built a complete React developer toolkit:

- **L1–L2:** React, Vite, JSX, components, ReactDOM
- **L3–L4:** State, props, hooks, events, `useEffect`, `useRef`
- **L5:** Controlled forms, validation, conditional rendering
- **L6:** Reusable components, project organization, refactoring
- **L7:** Data fetching, `async/await`, pessimistic & optimistic updates
- **L8:** Performance — `useMemo`, `useCallback`, `React.memo`
- **L9:** `useReducer`, `useContext`, combined state management
- **L10:** React Router, client-side navigation, protected routes
- **L11:** Portfolio polish, security, deployment

Final submission: Deploy your app (optional) and submit your **live URL + GitHub repo link** in CTD Learns. You're ready for Node.js and Express. Welcome to the community of React developers!